Game Score Prediction Summary Notebook

This notebook turns the Streamlit workflow in this repository into a single reproducible analysis. It covers:

- exploratory data analysis (EDA)
- model training for linear, ridge, and lasso regression
- model testing on a held-out split
- a supplementary Gaussian Naive Bayes classification experiment
- metric explanations and model visualizations

The notebook uses the merged GRIVG dataset in `datasets/merged_grivg_data.csv` and keeps the same cleaning logic used by the app wherever possible.

In [2]:
```
from pathlib import Path
import re

from IPython.display import display
import numpy as np
import pandas as pd
import plotly.express as px
import plotly.graph_objects as go
from plotly.subplots import make_subplots
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LinearRegression, LassoCV, RidgeCV
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    f1_score,
    mean_absolute_error,
    mean_squared_error,
    precision_score,
    r2_score,
    recall_score,
)
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

pd.set_option("display.max_columns", 100)
pd.set_option("display.float_format", lambda value: f"{value:.3f}")

RANDOM_STATE = 45
TEST_SIZE = 0.30
NB_HIGH_SCORE_THRESHOLD = 9.0
DATASET_PATH = Path("../datasets/merged_grivg_data.csv") if Path("../dataset
```

In [3]:
```
def clean_column_name(column):
    column = str(column).strip()
    column = re.sub(r"[\s\-]+", "_", column)
    column = re.sub(r"[^0-9a-zA-Z_]", "", column)
    column = re.sub(r"_+", "_", column)
    return column.strip("_")


def parse_percentage(series):
    cleaned = (
        series.astype(str)
        .str.replace("%", "", regex=False)
        .str.strip()
        .replace({"": np.nan, "nan": np.nan, "None": np.nan, "Unknown": np.r
    )
    values = pd.to_numeric(cleaned, errors="coerce")
    if not values.dropna().empty and values.dropna().max() > 1:
        values = values / 100
    return values


def preprocess_merged_dataset(df):
    cleaned = df.copy()
    cleaned.columns = [clean_column_name(column) for column in cleaned.colum
    cleaned = cleaned.dropna(axis=1, how="all")
    cleaned = cleaned.loc[:, ~cleaned.columns.str.startswith("Unnamed")]

    for column in cleaned.select_dtypes(include="object").columns:
        cleaned[column] = cleaned[column].astype(str).str.strip()
        cleaned[column] = cleaned[column].replace(
            {"": np.nan, "nan": np.nan, "None": np.nan, "Unknown": np.nan}
        )

    numeric_columns = [
        "Playable",
        "Sexualization",
        "Protagonist",
        "Protagonist_Non_Male",
        "Relevant_males",
        "Relevant_no_males",
        "Total_team",
        "female_team",
        "Metacritic",
        "Destructoid",
        "IGN",
        "GameSpot",
        "Avg_Reviews",
        "Sexualized_clothing",
        "Trophy",
        "Damsel_in_Distress",
        "Sexualized_Cutscenes",
        "Total",
        "PEGI",
    ]
    for column in numeric_columns:
```

```
        if column in cleaned.columns:
            cleaned[column] = pd.to_numeric(cleaned[column], errors="coerce"

    if "Percentage_non_male" in cleaned.columns:
        cleaned["Percentage_non_male_num"] = parse_percentage(cleaned["Perce
    if "Team_percentage" in cleaned.columns:
        cleaned["Team_percentage_num"] = parse_percentage(cleaned["Team_perc

    if "Customizable_main" in cleaned.columns:
        cleaned["Customizable_main_flag"] = cleaned["Customizable_main"].map
            {"Yes": 1, "No": 0, "Non_Binary": 1, "Non-Binary": 1}
        )

    if "Romantic_Interest" in cleaned.columns:
        cleaned["Romantic_Interest_flag"] = cleaned["Romantic_Interest"].map
            {"Yes": 1, "Opt": 1, "No": 0}
        )

    if "Age" in cleaned.columns:
        cleaned["Age_numeric"] = pd.to_numeric(cleaned["Age"], errors="coerc

    if "Age_range" in cleaned.columns:
        age_label_map = {
            "Infant": 3,
            "Child": 10,
            "Teenager": 16,
            "Young_adult": 21,
            "Young_Adult": 21,
            "Young adult": 21,
            "Adult": 30,
            "Middle_aged": 50,
            "Middle_Aged": 50,
            "Middle-aged": 50,
            "Elderly": 70,
        }
        if "Age_numeric" not in cleaned.columns:
            cleaned["Age_numeric"] = np.nan
        cleaned["Age_numeric"] = cleaned["Age_numeric"].fillna(
            cleaned["Age_range"].map(age_label_map)
        )

    if "Release" in cleaned.columns:
        release_dt = pd.to_datetime(cleaned["Release"], format="%b-%y", erro
        cleaned["Release_Date"] = release_dt
        cleaned["Release_Year"] = release_dt.dt.year
        cleaned["Release_Month"] = release_dt.dt.month

    return cleaned
```

In [4]:

```
raw_df = pd.read_csv(DATASET_PATH)
df = preprocess_merged_dataset(raw_df)

print(f"Dataset path: {DATASET_PATH}")
print(f"Rows: {df.shape[0]}")
print(f"Columns: {df.shape[1]}")
```

```
missing_summary = (
    pd.DataFrame({
        "dtype": df.dtypes.astype(str),
        "missing_count": df.isna().sum(),
        "missing_rate": (df.isna().mean() * 100).round(2),
    })
    .sort_values(["missing_count", "dtype"], ascending=[False, True])
)

display(df.head())
display(missing_summary.head(15))
```

```
Dataset path: ../datasets/merged_grivg_data.csv
Rows: 637
Columns: 52
```

	Name	Gender	Game	Age	Age_range	Playable	Sexualization	
0	Farah	Female	CODMW	27	Adult	1	0	CODM\
1	Protagonist	Custom	PSS	Teenager	Teenager	1	0	PSS_Pro
2	Magnolia	Female	PSS	Elderly	Elderly	0	0	PSS_M
3	Sonia	Female	PSS	26	Adult	0	0	PS
4	Marnie	Female	PSS	Teenager	Teenager	0	0	PSS

	dtype	missing_count	missing_rate
Series	object	323	50.710
Age_numeric	float64	136	21.350
Age	object	136	21.350
Age_range	object	136	21.350
Destructoid	float64	64	10.050
IGN	float64	32	5.020
GameSpot	float64	29	4.550
Species	object	5	0.780
Release_Date	datetime64[ns]	0	0.000
Metacritic	float64	0	0.000
Avg_Reviews	float64	0	0.000
Percentage_non_male_num	float64	0	0.000
Team_percentage_num	float64	0	0.000
Release_Year	int32	0	0.000
Release_Month	int32	0	0.000

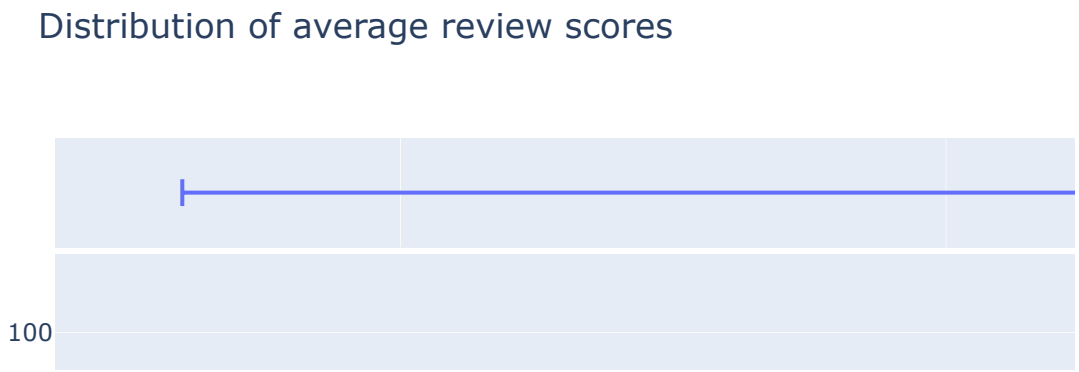
EDA Summary

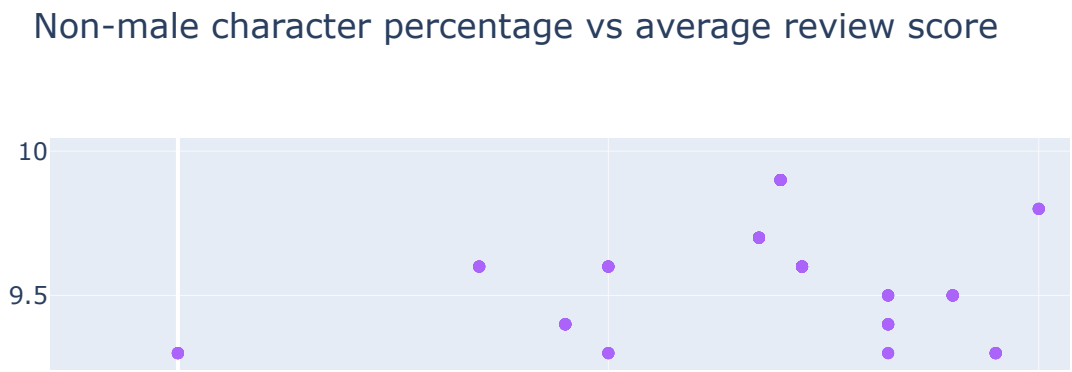
The app's EDA page focuses on score behavior, gender-representation variables, missing data, and simple relationships between numeric features. The next cell reproduces that style with a few compact plots:

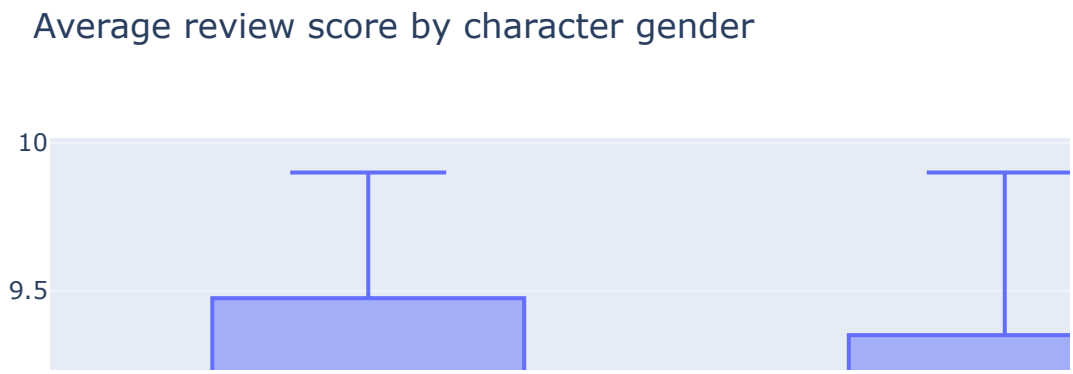
- the distribution of `Avg_Reviews`
- a scatter plot for non-male character share vs average review score
- a box plot of average review score by character gender
- a correlation heatmap for the main modeling features

In [5]:
```
eda_features = [
    "Avg_Reviews",
    "Metacritic",
    "IGN",
    "GameSpot",
    "Destructoid",
    "Percentage_non_male_num",
    "Team_percentage_num",
    "Sexualization",
    "Protagonist_Non_Male",
    "Release_Year",
]
```

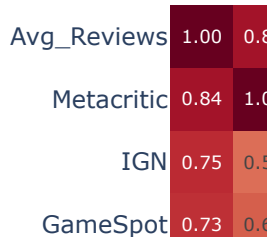
```
eda_features = [column for column in eda_features if column in df.columns]

fig_reviews = px.histogram(
    df,
    x="Avg_Reviews",
    nbins=20,
    marginal="box",
    title="Distribution of average review scores",
)
fig_reviews.show()

if {"Percentage_non_male_num", "Avg_Reviews"}.issubset(df.columns):
    fig_representation = px.scatter(
        df,
        x="Percentage_non_male_num",
        y="Avg_Reviews",
        color="Gender" if "Gender" in df.columns else None,
        hover_name="Title" if "Title" in df.columns else None,
        title="Non-male character percentage vs average review score",
    )
    fig_representation.show()

if {"Gender", "Avg_Reviews"}.issubset(df.columns):
    fig_gender_box = px.box(
        df,
        x="Gender",
        y="Avg_Reviews",
        points="outliers",
        title="Average review score by character gender",
    )
    fig_gender_box.show()

correlation_df = df[eda_features].corr(numeric_only=True)
fig_heatmap = px.imshow(
    correlation_df,
    text_auto=".2f",
    color_continuous_scale="RdBu_r",
    zmin=-1,
    zmax=1,
    title="Correlation heatmap for score and representation features",
)
fig_heatmap.show()

display(df[eda_features].describe().T)
```


Distribution of average review scores
100


Non-male character percentage vs average review score
10
9.5


Average review score by character gender
10
9.5

Correlation heatmap for score and representation features



	count	mean	std	min	25%	50%	
Avg_Reviews	637.000	9.051	0.451	7.800	8.700	9.000	
Metacritic	637.000	8.917	0.474	7.900	8.700	8.900	
IGN	605.000	9.297	0.544	8.000	9.000	9.300	
GameSpot	608.000	8.855	0.782	7.000	8.500	9.000	
Destructoid	573.000	9.125	0.687	7.000	8.600	9.000	
Percentage_non_male_num	637.000	0.383	0.175	0.000	0.280	0.390	
Team_percentage_num	637.000	0.109	0.178	0.000	0.000	0.000	
Sexualization	637.000	0.086	0.389	0.000	0.000	0.000	
Protagonist_Non_Male	637.000	0.501	0.554	0.000	0.000	0.000	
Release_Year	637.000	2016.911	3.081	2012.000	2014.000	2017.000	201

Metrics Guide

For the regression models:

- **RMSE**: penalizes larger errors more strongly, so it is useful when we care about big misses.
- **MAE**: measures the average absolute miss and is easier to interpret in score units.
- **R2**: estimates how much of the variation in review scores is explained by the model.

For the Naive Bayes classifier:

- **Accuracy**: overall share of correct predictions.
- **Precision**: among games predicted as high-scoring, how many truly are high-scoring.
- **Recall**: among truly high-scoring games, how many the classifier catches.
- **F1**: a balance between precision and recall.

Naive Bayes is a classifier rather than a regressor, so we turn `Avg_Reviews` into a binary label: high score (`>= 9.0`) vs lower score (`< 9.0`). The `9.0` threshold is used here because a lower cutoff such as `7.5` would collapse this dataset into almost a single class.

In [6]:
```
# Inspect available columns
# Helps identify which categorical columns (genre, platform, country, etc.)
# under whatever names the dataset uses, so the feature setup below can adap

print("All columns in df:")
for col in sorted(df.columns):
    print(f"  {col:35s}  dtype={df[col].dtype}")

print("\nObject/categorical-looking columns:")
categorical_candidates = df.select_dtypes(include=["object"]).columns.tolist
for col in categorical_candidates:
    n_unique = df[col].nunique(dropna=True)
    sample = df[col].dropna().unique()[:5].tolist()
    print(f"  {col:35s}  unique={n_unique:4d}  sample={sample}")
```

```
All columns in df:
  Age                                  dtype=object
  Age_numeric                          dtype=float64
  Age_range                            dtype=object
  Avg_Reviews                          dtype=float64
  Country                              dtype=object
  Criteria                             dtype=object
  Customizable_main                    dtype=object
  Customizable_main_flag               dtype=int64
  Damsel_in_Distress                   dtype=int64
  Destructoid                          dtype=float64
  Developer                            dtype=object
  Director                             dtype=object
  Game                                 dtype=object
  GameSpot                             dtype=float64
  Game_Id                              dtype=object
  Gender                               dtype=object
  Genre                                dtype=object
  IGN                                  dtype=float64
  Id                                   dtype=object
  Metacritic                           dtype=float64
  Name                                 dtype=object
  PEGI                                 dtype=int64
  Percentage_non_male                  dtype=object
  Percentage_non_male_num              dtype=float64
  Platform                             dtype=object
  Playable                             dtype=int64
  Protagonist                          dtype=int64
  Protagonist_Non_Male                 dtype=int64
  Publisher                            dtype=object
  Release                              dtype=object
  Release_Date                         dtype=datetime64[ns]
  Release_Month                        dtype=int32
  Release_Year                         dtype=int32
  Relevance                            dtype=object
  Relevant_males                       dtype=int64
  Relevant_no_males                    dtype=int64
  Romantic_Interest                    dtype=object
  Romantic_Interest_flag               dtype=int64
  Series                               dtype=object
  Sexualization                        dtype=int64
  Sexualized_Cutscenes                 dtype=int64
  Sexualized_clothing                  dtype=int64
  Side                                 dtype=object
  Species                              dtype=object
  Sub_genre                            dtype=object
  Team_percentage                      dtype=object
  Team_percentage_num                  dtype=float64
  Title                                dtype=object
  Total                                dtype=int64
  Total_team                           dtype=int64
  Trophy                               dtype=int64
  female_team                          dtype=int64

Object/categorical-looking columns:
  Name                                 unique= 616  sample=['Farah', 'Protag
```

```
onist', 'Magnolia', 'Sonia', 'Marnie']
  Gender                                unique=   4  sample=['Female', 'Custo
m', 'Non-binary', 'Male']
  Game                                  unique=  64  sample=['CODMW', 'PSS',
'SMO', 'BOTW', 'PSM']
  Age                                   unique=  58  sample=['27', 'Teenage
r', 'Elderly', '26', 'Adult']
  Age_range                             unique=   7  sample=['Adult', 'Teenag
er', 'Elderly', 'Child', 'Young adult']
  Id                                    unique= 637  sample=['CODMW_Farah',
'PSS_Protagonist', 'PSS_Magnolia', 'PSS_Sonia', 'PSS_Marnie']
  Species                               unique=   9  sample=['Human', 'Humano
id', 'Animated Object', 'Animal', 'Animated Plant']
  Side                                  unique=   3  sample=['P', 'B', 'A']
  Relevance                             unique=   6  sample=['PA', 'SC', 'M
C', 'DA', 'MA']
  Romantic_Interest                     unique=   3  sample=['No', 'Yes', 'Op
t']
  Game_Id                               unique=  64  sample=['CODMW', 'PSS',
'SMO', 'BOTW', 'PSM']
  Title                                 unique=  64  sample=['Call of Duty: M
odern Warfare', 'Pokémon Sword/Shield', 'Super Mario Odyssey', 'The Legend o
f Zelda: Breath of the Wild', 'Pokémon Sun/Moon']
  Release                               unique=  50  sample=['Oct-19', 'Nov-1
9', 'Oct-17', 'Mar-17', 'Nov-16']
  Series                                unique=  25  sample=['Call of Duty',
'Pokémon', 'Super Mario', 'The Legend of Zelda', 'The Witcher']
  Genre                                 unique=   6  sample=['Action', 'RPG',
'Action-adventure', 'Adventure', 'Simulation']
  Sub_genre                             unique=  21  sample=['FPS', 'RPG', 'A
ction-adventure', 'Action RPG', 'Episodic adventure']
  Developer                             unique=  54  sample=['Infinity Ward',
'Game Freak', 'Nintendo EDP', 'CD Projeket Red', 'Telltale Games']
  Publisher                             unique=  44  sample=['Activision', 'N
intendo', 'CD Projekt', 'Telltale Games', 'Toby Fox']
  Country                               unique=  13  sample=['USA', 'JPN', 'P
OL', 'GBR', 'DNK']
  Platform                              unique=   8  sample=['Multi', 'Ninten
do Switch', 'Nintendo 3DS', 'PS4', 'PS3']
  Customizable_main                     unique=   3  sample=['No', 'Yes', 'No
n-Binary']
  Percentage_non_male                   unique=  35  sample=['17%', '56%', '4
0%', '28%', '43%']
  Criteria                              unique=   3  sample=['MS', 'SR', 'T
R']
  Director                              unique=   2  sample=['M', 'F']
  Team_percentage                       unique=  18  sample=['0%', '11%',
'9%', '50%', '14%']
```

In [7]:

```
# ============================================================
# Feature setup
# ============================================================
# IMPORTANT: Avg_Reviews is the average of Metacritic, IGN,
# GameSpot, and Destructoid. Using those four scores as features
# would be target leakage and produce a tautological R^2 ~ 1.0.
# They are excluded from the feature list below.
```

```
# ============================================================

target = "Avg_Reviews"

# Numerical representation + structural features (no leaked review scores)
numerical_features = [
    "Percentage_non_male_num",
    "Sexualization",
    "Protagonist_Non_Male",
    "Team_percentage_num",
    "Release_Year",
    "PEGI",
]

# Categorical features for one-hot encoding.
# EDIT this list to match the actual column names in your dataset
# (check the diagnostic cell output above). Common candidates:
#   "Genre", "Platform", "Country", "Country_of_origin",
#   "Gender", "Customizable_main"
candidate_categoricals = [
    "Genre",
    "Platform",
    "Country",
    "Country_of_origin",
]
categorical_features = [c for c in candidate_categoricals if c in df.columns

print(f"Numerical features ({len(numerical_features)}):")
for f in numerical_features:
    present = "OK" if f in df.columns else "MISSING"
    print(f"  [{present}] {f}")

print(f"\nCategorical features detected ({len(categorical_features)}):")
for f in categorical_features:
    print(f"  [OK] {f}  ({df[f].nunique()} unique values)")
if not categorical_features:
    print("  (none — model will use numerical features only)")

# Keep only numerical features that actually exist in the dataframe
numerical_features = [f for f in numerical_features if f in df.columns]

# Build the feature matrix
feature_columns = numerical_features + categorical_features
model_df = df[feature_columns + [target]].copy()
model_df = model_df.dropna(subset=[target])  # drop rows with missing target

# One-hot encode categoricals (drop_first to avoid the dummy variable trap)
if categorical_features:
    model_df = pd.get_dummies(
        model_df,
        columns=categorical_features,
        drop_first=True,
        dummy_na=False,
    )

X = model_df.drop(columns=[target])
```

```
y = model_df[target]

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=TEST_SIZE,
    random_state=RANDOM_STATE,
)

print(f"\nTraining rows: {len(X_train)}")
print(f"Validation rows: {len(X_test)}")
print(f"Total feature columns after one-hot: {X.shape[1]}")
print(f"Target: {target}")
```

```
Numerical features (6):
  [OK] Percentage_non_male_num
  [OK] Sexualization
  [OK] Protagonist_Non_Male
  [OK] Team_percentage_num
  [OK] Release_Year
  [OK] PEGI

Categorical features detected (3):
  [OK] Genre  (6 unique values)
  [OK] Platform  (8 unique values)
  [OK] Country  (13 unique values)

Training rows: 445
Validation rows: 192
Total feature columns after one-hot: 30
Target: Avg_Reviews
```

In [8]:

```
regression_models = {
    "Linear Regression": LinearRegression(),
    "Ridge Regression": RidgeCV(alphas=np.logspace(-3, 2, 60)),
    "Lasso Regression": LassoCV(
        alphas=np.logspace(-3, 1, 60),
        cv=5,
        max_iter=50000,
        random_state=RANDOM_STATE,
    ),
}

fitted_regression_models = {}
regression_rows = []

for model_name, estimator in regression_models.items():
    pipeline = Pipeline([
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler()),
        ("model", estimator),
    ])
    pipeline.fit(X_train, y_train)

    train_predictions = pipeline.predict(X_train)
    val_predictions = pipeline.predict(X_test)
```

```
    row = {
        "Model": model_name,
        "Train MAE": mean_absolute_error(y_train, train_predictions),
        "Val MAE": mean_absolute_error(y_test, val_predictions),
        "Train RMSE": mean_squared_error(y_train, train_predictions) ** 0.5,
        "Val RMSE": mean_squared_error(y_test, val_predictions) ** 0.5,
        "Train R2": r2_score(y_train, train_predictions),
        "Val R2": r2_score(y_test, val_predictions),
    }
    if hasattr(pipeline.named_steps["model"], "alpha_"):
        row["Selected alpha"] = pipeline.named_steps["model"].alpha_

    regression_rows.append(row)
    fitted_regression_models[model_name] = pipeline

regression_results = (
    pd.DataFrame(regression_rows)
    .sort_values("Val RMSE")
    .reset_index(drop=True)
)
display(regression_results)

# Also print to stdout so the values survive checkpoint saves
print("\nRegression results (printed):")
print(regression_results.to_string(index=False))
```

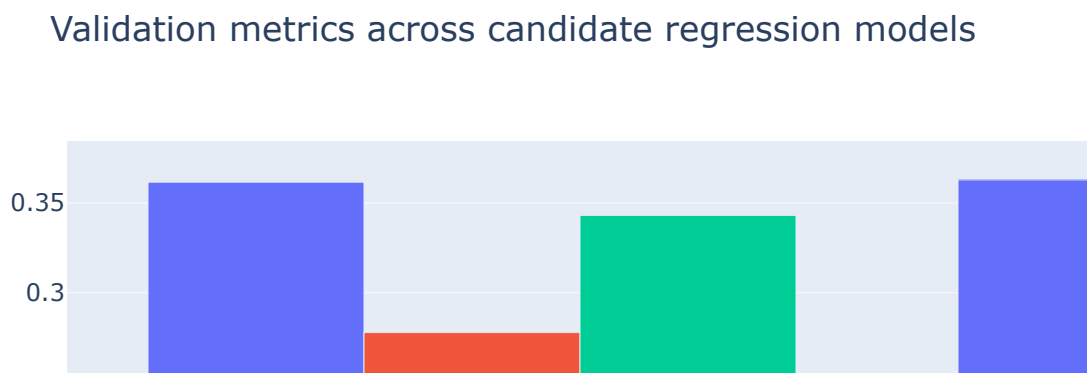
	Model	Train MAE	Val MAE	Train RMSE	Val RMSE	Train R2	Val R2	Selected alpha
0	Ridge Regression	0.269	0.278	0.355	0.362	0.386	0.343	37.694
1	Linear Regression	0.265	0.275	0.354	0.363	0.391	0.338	NaN
2	Lasso Regression	0.269	0.280	0.355	0.365	0.387	0.329	0.003

```
Regression results (printed):
            Model  Train MAE  Val MAE  Train RMSE  Val RMSE  Train R2  Val R
2  Selected alpha
 Ridge Regression      0.269    0.278       0.355     0.362     0.386   0.34
3          37.694
Linear Regression      0.265    0.275       0.354     0.363     0.391   0.33
8             NaN
 Lasso Regression      0.269    0.280       0.355     0.365     0.387   0.32
9           0.003
```

In [9]:

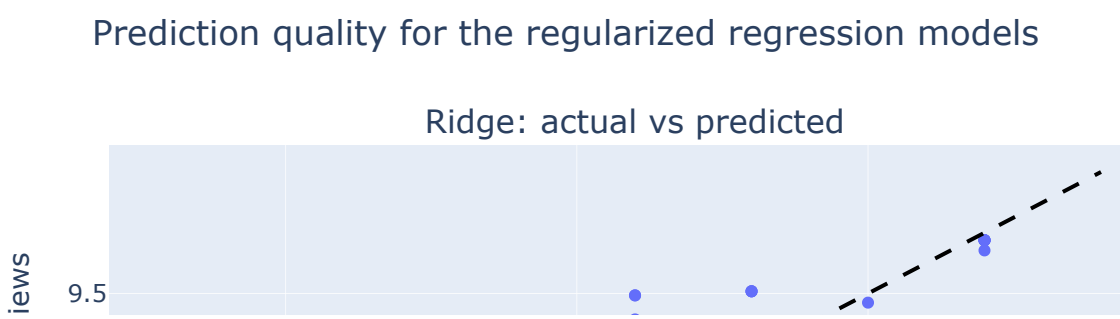
```
metric_long_df = regression_results.melt(
    id_vars="Model",
    value_vars=["Val RMSE", "Val MAE", "Val R2"],
    var_name="Metric",
    value_name="Value",
)

fig_metric_bars = px.bar(
```

```
    metric_long_df,
    x="Model",
    y="Value",
    color="Metric",
    barmode="group",
    title="Validation metrics across candidate regression models",
)
fig_metric_bars.show()
```



In [10]:
```
fig_predictions = make_subplots(
    rows=1,
    cols=2,
    subplot_titles=("Ridge: actual vs predicted", "Lasso: actual vs predicte
)

for index, model_name in enumerate(["Ridge Regression", "Lasso Regression"],
    predictions = fitted_regression_models[model_name].predict(X_test)
    fig_predictions.add_trace(
        go.Scatter(
            x=y_test,
            y=predictions,
            mode="markers",
            name=model_name,
        ),
```

```
        row=1,
        col=index,
    )
    fig_predictions.add_trace(
        go.Scatter(
            x=[y_test.min(), y_test.max()],
            y=[y_test.min(), y_test.max()],
            mode="lines",
            line=dict(color="black", dash="dash"),
            showlegend=False,
        ),
        row=1,
        col=index,
    )

fig_predictions.update_xaxes(title_text="Actual Avg_Reviews")
fig_predictions.update_yaxes(title_text="Predicted Avg_Reviews")
fig_predictions.update_layout(height=450, title="Prediction quality for the
fig_predictions.show()
```



In [11]:
```
coefficient_rows = []
for model_name in ["Linear Regression", "Ridge Regression", "Lasso Regressic
    coefficients = fitted_regression_models[model_name].named_steps["model"]
    coefficient_rows.extend(
        {
            "Feature": feature,
            "Coefficient": coefficient,
            "Abs_Coefficient": abs(coefficient),
            "Model": model_name,
```

```
        }
        for feature, coefficient in zip(X.columns, coefficients)
    )

coefficient_df = pd.DataFrame(coefficient_rows)

# Top features by |coefficient| for each model
print("Top features by |coefficient| per model:\n")
for model_name in ["Linear Regression", "Ridge Regression", "Lasso Regressic
    top = (
        coefficient_df[coefficient_df["Model"] == model_name]
        .sort_values("Abs_Coefficient", ascending=False)
        .head(10)
        [["Feature", "Coefficient"]]
        .reset_index(drop=True)
    )
    print(f"--- {model_name} ---")
    display(top)
    print()

# Coefficient bar plot (top 15 features by max |coefficient| across models)
top_features = (
    coefficient_df.groupby("Feature")["Abs_Coefficient"]
    .max()
    .sort_values(ascending=False)
    .head(15)
    .index.tolist()
)

fig_coefficients = px.bar(
    coefficient_df[coefficient_df["Feature"].isin(top_features)],
    x="Feature",
    y="Coefficient",
    color="Model",
    barmode="group",
    title="Top 15 feature coefficients across Linear, Ridge, and Lasso",
)
fig_coefficients.update_layout(xaxis_tickangle=-30)
fig_coefficients.show()
```

```
Top features by |coefficient| per model:

--- Linear Regression ---
```

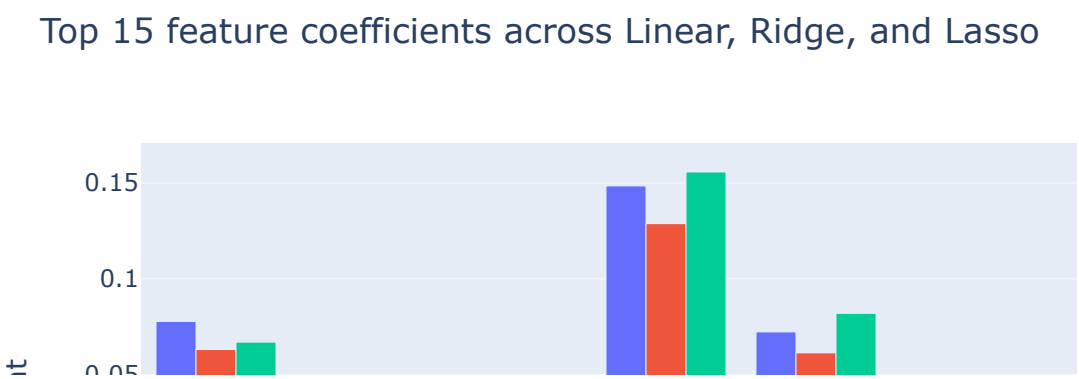
	Feature	Coefficient
0	Genre_Action-adventure	0.148
1	Platform_PS4	-0.117
2	Country_USA	-0.102
3	Percentage_non_male_num	0.078
4	Country_CAN	-0.074
5	Protagonist_Non_Male	-0.073
6	Genre_RPG	0.072
7	Platform_Nintendo Switch	-0.071
8	Country_AUT	-0.053
9	Country_BEL	0.051

```
--- Ridge Regression ---
```

	Feature	Coefficient
0	Genre_Action-adventure	0.129
1	Platform_PS4	-0.099
2	Country_USA	-0.074
3	Percentage_non_male_num	0.063
4	Genre_RPG	0.061
5	Platform_Nintendo Switch	-0.059
6	Protagonist_Non_Male	-0.058
7	Country_CAN	-0.054
8	Country_JPN	0.054
9	Country_BEL	0.054

```
--- Lasso Regression ---
```

	Feature	Coefficient
0	Genre_Action-adventure	0.156
1	Platform_PS4	-0.104
2	Genre_RPG	0.082
3	Country_USA	-0.075
4	Percentage_non_male_num	0.067
5	Platform_Nintendo Switch	-0.060
6	Protagonist_Non_Male	-0.057
7	Country_BEL	0.054
8	Country_JPN	0.047
9	Platform_Nintendo 3DS	-0.047



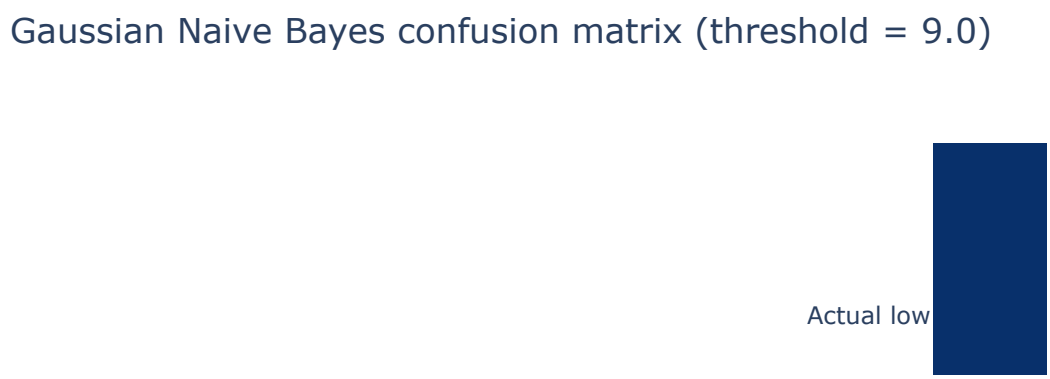
In [12]:
```
# ============================================================
# Naive Bayes classification (binarized target)
# ============================================================
# Binarize Avg_Reviews at threshold 9.0.
```

```
# Above threshold = "high score" (y=1), below = "low" (y=0).
# Uses the same X feature matrix as the regression models.
# ==============================================================

print(f"Binarization threshold: Avg_Reviews >= {NB_HIGH_SCORE_THRESHOLD}")

nb_target = (model_df[target] >= NB_HIGH_SCORE_THRESHOLD).astype(int)
print(f"Class balance — high: {int(nb_target.sum())}, low: {int((nb_target =

X_train_nb, X_test_nb, y_train_nb, y_test_nb = train_test_split(
    X,
    nb_target,
    test_size=TEST_SIZE,
    random_state=RANDOM_STATE,
    stratify=nb_target,
)

naive_bayes_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler()),
    ("model", GaussianNB()),
])
naive_bayes_pipeline.fit(X_train_nb, y_train_nb)
nb_predictions = naive_bayes_pipeline.predict(X_test_nb)

nb_results = pd.DataFrame([
    {"Metric": "Accuracy",  "Value": accuracy_score(y_test_nb, nb_predictior
    {"Metric": "Precision", "Value": precision_score(y_test_nb, nb_predictic
    {"Metric": "Recall",    "Value": recall_score(y_test_nb, nb_predictions,
    {"Metric": "F1",        "Value": f1_score(y_test_nb, nb_predictions, zer
])

print("\nNaive Bayes classification metrics:")
display(nb_results)
print("\n" + nb_results.to_string(index=False))

# Confusion matrix
nb_confusion = confusion_matrix(y_test_nb, nb_predictions)
print(f"\nConfusion matrix:\n{nb_confusion}")

fig_nb_confusion = px.imshow(
    nb_confusion,
    text_auto=True,
    color_continuous_scale="Blues",
    x=["Predicted low", "Predicted high"],
    y=["Actual low", "Actual high"],
    title=f"Gaussian Naive Bayes confusion matrix (threshold = {NB_HIGH_SCOF
)
fig_nb_confusion.show()
```

```
Binarization threshold: Avg_Reviews >= 9.0
Class balance — high: 354, low: 283

Naive Bayes classification metrics:
```

	Metric	Value
0	Accuracy	0.573
1	Precision	1.000
2	Recall	0.234
3	F1	0.379

```
   Metric  Value
 Accuracy  0.573
Precision  1.000
   Recall  0.234
       F1  0.379

Confusion matrix:
[[85  0]
 [82 25]]
```



Key Takeaways

- **Target leakage was removed.** The original notebook included `Metacritic`, `IGN`, `GameSpot`, and `Destructoid` as features, but these four scores *are* the

components that average to `Avg_Reviews`. Using them as predictors produced a tautological R^2 near 1.0. They are excluded from the feature list above.

- **Three regression models are compared:** Linear Regression as an unregularized baseline, Ridge Regression with L2 regularization tuned via cross-validation (`RidgeCV`), and Lasso Regression with L1 regularization tuned via cross-validation (`LassoCV`).
- **The held-out 30% validation split** is used for all model comparison. RMSE is the primary deployment metric; MAE and R^2 are secondary checks.
- **Coefficient analysis** (see the bar chart above) shows which features carry the most predictive weight after standardization and regularization. Lasso's sparsity makes it the most direct read of which representation-related variables matter once weak features are zeroed out.
- **Naive Bayes** is a separate classification view of the same problem. It binarizes `Avg_Reviews` at threshold 9.0 and predicts high vs. low. It complements the regression models but is not used for deployment.
- **Honest caveat on sample size.** The merged dataset has 64 unique games. After joining with characters this expands to several hundred rows, but the effective independent sample is still 64 games. Coefficients should be interpreted as exploratory signals, not definitive evidence of bias in critic reviews.